

System-Level Integration & Verification: UART-to-ALU Bridge

Engineer: Waleed Emad Yahyaa

Course: SystemVerilog for Design

Project Overview

This project implements a complete SoC-style communication bridge. It enables a remote user to perform memory-mapped I/O (Register File) and computational tasks (ALU) via a UART interface. The design focuses on robust **Clock Domain Crossing (CDC)**, **Power Management**, and **Deep Hierarchical Verification**.

⚡ Advanced SystemVerilog Features Implemented

The following SV-specific features were utilized to ensure hardware efficiency and verification clarity:

1. Functional Modeling & Logic Types

- **logic Type:** Moved away from the ambiguous `reg/wire` distinction. Used `logic` throughout the design to prevent multi-driver issues and improve compilation intent.
- **always_ff & always_comb:** Employed specialized procedural blocks to ensure synthesis tools correctly infer sequential and combinational logic, reducing the risk of unintended latches.

2. Design Hierarchy & Connectivity

- **SV Packages (SYS_PACKAGE.sv):** Encapsulated shared parameters and constants. This ensures "Single Source of Truth," making the design modular and preventing parameter mismatch across the UART and System Controller.
- **Port Mapping:** Used SV's concise port connection styles to keep the top-level integration (`top_sys.sv`) clean and readable.

3. FSM Architecture

- **Enumerated Types (enum):** Defined FSM states (IDLE, CMD, ADDR, etc.) using `enum`. This provides human-readable state names in the Waveform Viewer (Questasim) instead of raw hex values, significantly speeding up the debug process.

4. Advanced Verification (Testbench)

- **Tasks & Functions:** Created a high-level "Bus Functional Model" (BFM) using tasks like `SEND_WR_CMD` and `CHECK_ALU`. This abstracts the bit-banging of the UART protocol away from the test sequence.
- **Enhanced String Formatting:** Leveraged `%0t` and `$display` with string concatenation to generate a dynamic, time-stamped verification report.

- **Hierarchical References:** Utilized dot-notation (e.g., DUT.u_uart_tx.i_CLK) to monitor internal signals for white-box testing without cluttering the top-level port list.
-

System Architecture

The design is partitioned into two main clock domains linked by an **Asynchronous FIFO**:

1. **UART Domain:** Handles serial-to-parallel conversion and parity checking.
2. **System Domain:** Operates the 16-operation ALU and the 16x8-bit Register File.

Special Features:

- **Clock Gating:** Uses a specialized clock gating cell to disable the ALU clock when idle, reducing dynamic power consumption.
 - **Prescale Multiplier:** A custom-built clock divider that dynamically adjusts the oversampling rate (8, 16, or 32) based on Register File configuration.
-

How to Run

The project is fully automated using a Questasim do script.

1. Open Questasim.
2. Navigate to the project directory.
3. Run the following command:

```
Tcl
do run.do
```

Simulation Features:

- **Deep Debug Mode:** The script automatically organizes waves into nested groups (UART, FIFO, ALU) for easy inspection.
 - **Automated Scoreboard:** The transcript will print a final summary indicating the number of test cases passed/failed.
-

Final Verification Result

The testbench concludes with a high-visibility report:

- **Total Testcases:** 71
 - **Passed:** 71
 - **Failed:** 0
 - **Status:** Mabroooooooooook! You Are Winner.
-

TA Note

The code is written with a "Design for Synthesis" mindset. All SV constructs used are synthesis-friendly and adhere to industry-standard coding styles (e.g., no logic in the clock path, synchronized resets).
